LeetCode Practice

12 Easy: Bit Manipulation

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

12
EASY

Patterns (in order)

1. Bit Manipulation (12)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

12 questions | tap a title to jump to it

Easy questions (12) Go >

Bit Manipulation (12) Go >

E 1. Add Binary LC 67
E 2. Single Number LC 136
E 3. Reverse Bits LC 190
E 4. Number of 1 Bits LC 191
E 5. Count Complete Tree Nodes LC 222
E 6. Power of Two LC 231
E 7. Missing Number LC 268
E 8. Counting Bits LC 338
E 9. Hamming Distance LC 461
E 10. Set Mismatch LC 645
E 11. Longest Nice Substring LC 1763
E 12. Sum of All Subset XOR Totals LC 1863

Easy questions (12)

EASY | TOPIC 1 OF 1

Bit Manipulation

12 Easy questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Add Binary LC 67
2. Single Number LC 136
3. Reverse Bits LC 190
4. Number of 1 Bits LC 191
5. Count Complete Tree Nodes LC 222
6. Power of Two LC 231
7. Missing Number LC 268
8. Counting Bits LC 338
9. Hamming Distance LC 461
10. Set Mismatch LC 645
11. Longest Nice Substring LC 1763
12. Sum of All Subset XOR Totals LC 1863

EASY | Bit Manipulation | LeetCode #67 | 1 of 12 | 1/12

Add Binary

QUESTION

Given two binary strings a and b, return their sum as a binary string.

Example 1	
Input	`a = "11", b = "1"`
Output	`"100"`

Example 2	
Input	`a = "1010", b = "1011"`
Output	`"10101"`

CONSTRAINTS

- 1 <= a.length, b.length <= 10^4
- a and b consist only of '0' or '1' characters.
- Each string does not contain leading zeros except for the zero itself.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given two binary strings a and b, return their sum as a binary string. Right?"

"A few quick questions:

Can they have leading zeros? No — except '0' itself. Can they be '0'? Yes."

"Let me walk: a='11', b='1' -> 11+01=100 -> '100' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
x = int(a, 2)
y = int(b, 2)
return bin(x + y) without the prefix
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Fixed-width integers overflow on long strings (and the problem wants string handling). Adding digit by digit from the right with a carry is O(max(n, m)) and uses no big numbers.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is big-int conversion.

Simulate binary addition from right to left with a carry variable:

At each step: total = a_bit + b_bit + carry. digit = total % 2. carry = total // 2.

Time: O(max(n,m)). Space: O(max(n,m)) for the result."

Time: O(n) — optimal single-pass solution. Linear time is best achievable.

Key insight: bit-by-bit addition from right to left with a carry variable mirrors the grade-school binary addition algorithm — O(max(a,b)) with no conversion.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _67_AddBinary:
    def addBinary(self, a, b):
        i, j, carry = len(a)-1, len(b)-1, 0
        result = []
        while i >= 0 or j >= 0 or carry:
            total = carry
            if i >= 0: total += int(a[i]); i -= 1
            if j >= 0: total += int(b[j]); j -= 1
            carry, digit = divmod(total, 2)
            result.append(str(digit))
        return ''.join(reversed(result))
```

PHASE 5 TEST & DEBUG

"Let me trace addBinary('11','1')."

Align from right. carry=0. result=[].

i=1 (a[1]='1'), j=0 (b[0]='1'): sum=1+1+0=2. bit=0, carry=1. result=[0].

i=0 (a[0]='1'): sum=1+0+1=2. bit=0, carry=1. result=[0,0].

carry=1: bit=1. result=[0,0,1].

Reverse: result='100' ok (3 in binary).

WHY use carry variable: binary addition handles each bit position plus carry-in independently.

'1'+'1': i=0,j=0: 1+1+0=2->bit=0,carry=1. carry=1->bit=1. Reverse->'10' ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(max(n,m)) process each bit once	O(max(n,m)) for result

Same pattern as Add Two Numbers (#2) — carry propagation from right to left.

Follow-up: Multiply Strings (#43) — O(n*m) positional multiplication.

Follow-up: Sum of Two Integers (#371) — use XOR for sum, AND for carry (bitwise only).

IN YOUR APP | Expose | Editorial | Community

EASY | Bit Manipulation | LeetCode #136 | 2 of 12 | 2/12

Single Number

QUESTION

Given a non-empty array of integers nums, every element appears twice except for one. Find that single one.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1	
Input	`nums = [2,2,1]`
Output	`1`

Example 2	
Input	`nums = [4,1,2,1,2]`
Output	`4`

Example 3	
Input	`nums = [1]`
Output	`1`

CONSTRAINTS

- $1 <= nums.length <= 3 * 10^4$
- $3 * 10^4 <= nums[i] <= 3 * 10^4$
- Each element in the array appears twice except for one element which appears only once.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Every element appears twice except one. Find that single element. O(n) time, O(1) space. Right?"

"A few quick questions:

Can the single element be negative? Yes. Array has at least one element? Yes."

"Let me walk: nums=[2,2,1] -> 1 ok. nums=[4,1,2,1,2] -> 4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for element in elements:
    count how many times it occurs by scanning
        the array
return the one that occurs once
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

A full scan per element. XOR of all numbers cancels pairs and leaves the single one: O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"XOR all elements — O(n). Walk-through:"

`result = 0.`

For each num in nums: result ^= num.

Return result.

HOW it works:

XOR is commutative and associative: order doesn't matter.

a ^ a = 0 (any number XOR itself = 0).

a ^ 0 = a (any number XOR 0 = itself).

All paired numbers cancel to 0. The single number has no pair -> remains.

Example: [4,1,2,1,2] -> 4^1^2^1^2 = 4^(1^1)^(2^2) = 4^0^0 = 4 ok

Time: O(n). Space: O(1)."

Key insight: XOR of two identical values is 0, and XOR is commutative and associative — all duplicates cancel in one pass, leaving only the unique element.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _136_SingleNumber:
    def singleNumber(self, nums):
        result = 0
        for num in nums:
            result ^= num
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace singleNumber([4,1,2,1,2])."

XOR all numbers: 4^1^2^1^2.

1^1=0. 2^2=0. 4^0=4.

XOR is commutative/associative: pairs cancel, single remains.

result=4 ok

[2,2,1]: 2^2=0. 0^1=1. Return 1 ok

WHY XOR: a^a=0, a^0=a. Pairs eliminate themselves. Only unpaired element survives.

`singleNumber([1]): XOR nothing (only itself) -> return 1 ok`

Why XOR: XOR is associative/commutative -> order doesn't matter -> pairs cancel regardless of position.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
single pass	one variable

XOR is the canonical trick for cancelling duplicate pairs in O(1) space.

Follow-up: Single Number II (#137) — every element appears 3 times except one.

Count bits: for each bit position, count appearances % 3 -> leftover = single.

Follow-up: Single Number III (#260) — two singles. XOR all->get XOR of the two.

Use rightmost set bit to split into two groups.

IN YOUR APP | Expose | Editorial | Community

EASY | Bit Manipulation | LeetCode #190 | 3 of 12 | 3/12

Reverse Bits

QUESTION

Reverse bits of a given 32 bits signed integer.

Example 1	
Input	n = 43261596
Output	964176192

Example 2	
Input	n = 2147483644
Output	1073741822

CONSTRAINTS

- 0 <= n <= 2^{31} - 2
- n is even.
- Follow up: If this function is called many times, how would you optimize it?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Reverse the 32-bit binary representation of an unsigned integer. Return the result. Right?"

"A few quick questions:

Always treat as 32-bit even if leading zeros? Yes.

Return as unsigned 32-bit integer? Yes."

"Let me walk: n=43261596 (binary: 00000010100101000001111010011100)

reversed: 00111001011110000010100101000000 -> 964176192 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
convert n to a 32-character binary string
reverse the string
parse it back to an integer
```

TIME	SPACE
O(1)	O(1)

WHY IT IS SLOW

Constant (32 bits), but the string round trip is wasteful. A loop that shifts bits from n into the result is the same O(1) with no strings; swapping halves of the word in log steps is faster still.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Approach: optimal approach

"The bottleneck is string allocation.

Bit-by-bit reversal with no string: for 32 iterations, extract LSB (Least Significant Bit) of n (n&1),

shift result left and OR in the extracted bit, then shift n right.

Time: O(32) = O(1). Space: O(1)."

Key insight: extracting and shifting bits one at a time (LSB -> result) is purely arithmetic — 32 iterations, no string allocation.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _190_ReverseBits:
    def reverseBits(self, n):
        result = 0
        for _ in range(32):
            result = (result << 1) | (n & 1)
            n >>= 1
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace reverseBits(0b00000010100101000001111010011100)."

n = 0b00000010100101000001111010011100 (=43261596).

Right-shift n, AND with 1 to get each bit. Build result by left-shifting result each step.

result starts at 0. For 32 bits: result = (result<<1) | (n&1). n>>=1.

bit 0 of n=0->result=0. bit 1=0->result=0. bit 2=1->result=1. bit 3=1->result=3. ...

After 32 steps: result = 0b00111001011110000010100101000000 = 964176192 ok

WHY 32 iterations: fixed 32-bit reverse (even if high bits are 0, they go to low end).

Edge: n=0 -> return 0. n=0b11111111111111111111111111111111 (all 1s) -> reversed = same -> 4294967295 ok

WHY build from LSB: bit[31-i]=bit[i] of n. Shifting n right and result left achieves this.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(32) =O(1)	O(1)

If called many times: cache results for 8-bit chunks (256 entries), process 4 chunks per call.

Follow-up: Number of 1 Bits (#191) — count set bits instead of reversing.

Follow-up: Power of Two (#231) — n>0 and n&(n-1)==0 checks exactly one set bit.

IN YOUR APP | Expose | Editorial | Community

EASY | Bit Manipulation | LeetCode #191 | 4 of 12 | 4/12

Number of 1 Bits

QUESTION

Given a positive integer n, write a function that returns the number of set bits in its binary representation (also known as the Hamming weight).

Example 1	
Input	n = 11
Output	3
Explanation	*The input binary string 1011 has a total of three set bits.*

Example 2	
Input	n = 128
Output	1
Explanation	*The input binary string 10000000 has a total of one set bit.*

Example 3	
Input	n = 2147483645
Output	30
Explanation	*The input binary string 11111111111111111111111111111101 has a total of thirty set bits.*

CONSTRAINTS

- $1 <= n <= 2^{31} - 1$
- Follow up: If this function is called many times, how would you optimize it?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the number of set bits (1s) in the binary representation of a positive integer.

Also called the Hamming weight. Right?"

"A few quick questions:

Treat as unsigned 32-bit? Yes. Can n be 0? Constraints say n>=1."

"Let me walk: n=11 (1011) -> 3 ok. n=128 (10000000) -> 1 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
count = 0
for positions in positionses:
    if that bit of n is 1:
        count = count + 1
```

TIME	SPACE
O(log n)	O(1)

WHY IT IS SLOW

Tests every bit position, even the zeros. n & (n - 1) clears the lowest set bit, so the loop runs once per set bit.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is iterating all 32 bits even for sparse inputs.

Brian Kernighan's trick: n & (n-1) clears the lowest set bit.

Count how many times we can clear a set bit until n=0.

Time: O(k) where k = number of set bits. Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _191_NumberOf1Bits:
    def hammingWeight(self, n):
        count = 0
        while n:
            n &= n - 1
            count += 1
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace hammingWeight(11=0b1011)."

Method 1: n & (n-1) clears lowest set bit. Count iterations until n=0.

n=1011: n&(n-1)=1011&1010=1010. count=1.

n=1010: n&(n-1)=1010&1001=1000. count=2. n=1000: n&(n-1)=0. count=3. Return 3 ok

Method 2: count = bin(n).count('1').

n=128=10000000: only 1 set bit. n&(n-1)=0 in 1 step -> count=1 ok.

WHY n&(n-1): flips the rightmost set bit to 0. Each iteration removes exactly one '1'. Count = Hamming weight.

n=0: no set bits -> return 0 ok. n=0xFFFFFFFF (all 1s): 32 iterations -> count=32 ok

WHY n&(n-1): n=1010->n-1=1001->AND=1000. Drops exactly one set bit per op. Faster than bit-by-bit.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(k) k = popcount, at most 32 iterations	**O(1)**

Brian Kernighan faster than 32-iteration scan when bits are sparse.

Built-in: bin(n).count('1') — valid in interviews.

Follow-up: Hamming Distance (#461) — popcount(x XOR y).

Follow-up: Counting Bits (#338) — popcount for all i in [0,n] using DP.

IN YOUR APP | Expose | Editorial | Community

EASY | Bit Manipulation | LeetCode #222 | 5 of 12 | 5/12

Count Complete Tree Nodes

QUESTION

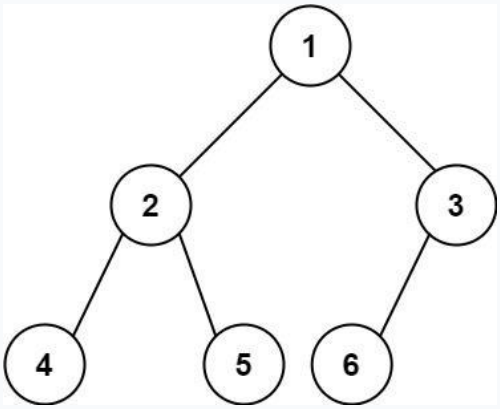
Given the root of a complete binary tree, return the number of the nodes in the tree.

According to Wikipedia, every level, except possibly the last, is completely filled in a complete binary tree, and all nodes in the last level are as far left as possible. It can have between 1 and 2^h nodes inclusive at the last level h.

Design an algorithm that runs in less than O(n) time complexity.

Example 1

Input	`root = [1,2,3,4,5,6]`
Output	`6`



Example 2

Input	`root = []`
Output	`0`

Example 3

Input	`root = [1]`
Output	`1`

CONSTRAINTS

- The number of nodes in the tree is in the range [0, $5 * 10^4$].
- $0 <= Node.val <= 5 * 10^4$
- The tree is guaranteed to be complete.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count the total number of nodes in a complete binary tree.

A complete binary tree has all levels full except possibly the last, filled left to right. Right?"

"A few quick questions:

Can we do better than O(n)? Yes — O(log²n) using binary search on the last level."

PHASE 1 CLARIFY

"Let me walk: root=[1,2,3,4,5,6] -> 6 nodes ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
count every node with a full DFS
```

TIME	SPACE
O(n)	O(h)

WHY IT IS SLOW

Ignores that the tree is complete. Compare the left-most and right-most depths: if equal the subtree is perfect (2^h - 1 nodes), else recurse: O(log^2 n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Approach: optimal approach

"The bottleneck is visiting every node.

For a complete tree: compare left-spine height vs right-spine height.

If equal -> left subtree is a perfect binary tree (2^h - 1 nodes).

If unequal -> right subtree is a perfect binary tree one level shorter.

Recurse into the non-perfect half.

Time: O(log²n). Space: O(log n)."

Key insight: in a complete binary tree, equal left/right spine depths mean the left subtree is perfect (2^h nodes) — this binary decision makes each step O(log n), total O(log²n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _222_CountCompleteTreeNodes:
    def countNodes(self, root):
        if not root:
            return 0
        left_h = right_h = 0
        left, right = root, root
        while left: left_h += 1; left = left.left
        while right: right_h += 1; right = right.right
        if left_h == right_h:
            return (1 << left_h) - 1
        return 1 + self.countNodes(root.left) + self.count
Nodes(root.right)
```

PHASE 5 TEST & DEBUG

"Let me trace the bit representation and show which bits get set/cleared and why."

root=[1,2,3,4,5,6]: left_h=3, right_h=2. Not equal -> recurse.

`countNodes(2): left_h=2,right_h=2 -> 2²-1=3.`
`countNodes(3): left_h=1,right_h=1 -> 1.`

1+3+1=5? Wait: root=1, left subtree=[2,4,5,null,null], right=[3,6].

Actually: left_h of 1: go left to 2 to 4 = 3. right_h of 1: go right to 3 to 6 to... wait.

Correct tree [1,2,3,4,5,6]: node 3 has left=6, right=None. right_h=2 (1->3). left_h=3 (1->2->4).

Unequal -> 1 + countNodes(2) + countNodes(3). Eventually -> 6 ok

The height comparison identifies perfect subtrees, enabling the skip

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log²n)	**O(log n)**
O(log n) levels, each computing heights in O(log n)	recursion depth

Follow-up: insert into a complete binary tree — same height trick to find the insertion point.

Follow-up: if tree is not complete, fall back to O(n) DFS.

EASY | Bit Manipulation | LeetCode #231 | 6 of 12 | 6/12

Power of Two

QUESTION

Given an integer n, return true if it is a power of two. Otherwise, return false.

An integer n is a power of two, if there exists an integer x such that n == 2^x.

Example 1

Input	`n = 1`
Output	`true`
Explanation	$2^0 = 1$

Example 2

Input	`n = 16`
Output	`true`
Explanation	$2^4 = 16$

Example 3

Input	`n = 3`
Output	`false`

CONSTRAINTS

- 2^{31} <= n <= 2^{31} - 1
- Follow up: Could you solve it without loops/recursion?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return true if n is a power of 2. n can be negative or zero. Right?"

"A few quick questions:

n=0 -> false. n=1 -> true (2^0). Negative -> false?"

"Let me walk: n=16 -> true ok. n=3 -> false ok. n=1 -> true ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while n is even:
    n = n / 2
return n == 1
```

TIME	SPACE
O(log n)	O(1)

NOTE

PHASE 2 BRUTE FORCE

Not slow (about 31 steps at most). A power of two has exactly one set bit, so n > 0 and n & (n - 1) == 0 is a single O(1) check.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Single bit check — O(1). Walk-through:"

A power of 2 in binary has exactly one '1' bit: 1=0001, 2=0010, 4=0100, 8=1000.

n & (n-1) clears the lowest set bit of n.

If n is a power of 2: n & (n-1) == 0 (the only '1' bit is cleared -> all zeros).

If n is NOT a power of 2: at least one other '1' bit remains -> result != 0.

Also check n > 0 (0 is not a power of 2, and 0&(-1) = 0 would incorrectly pass).

`return n > 0 and (n & (n-1)) == 0.`

Time: O(1). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _231_PowerOfTwo:
    def isPowerOfTwo(self, n):
        return n > 0 and (n & (n - 1)) == 0
```

PHASE 5 TEST & DEBUG

"Let me trace isPowerOfTwo(16) and isPowerOfTwo(6)."

16 in binary: 10000. n & (n-1) = 10000 & 01111 = 00000 = 0 -> True ok

6 in binary: 110. n & (n-1) = 110 & 101 = 100 != 0 -> False ok

WHY n & (n-1): powers of two have exactly one set bit; subtracting 1 flips that bit and all below.

Edge: n=1 -> 1 & 0 = 0 -> True (2=1 is a power of two) ok

Edge: n=0 -> not a power of two; n<0 -> False (negative never power of two).

Trace: n=8 (1000) -> 1000 & 0111 = 0 -> True ok

Trace: n=32 (100000) -> 32&31=100000&011111=0 -> True ok. n=12 (1100) -> 1100&1011=1000!=0 -> False ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(1)	O(1)

Alternative: n>0 and n==(n & -n) (isolate lowest set bit and check equals n).

Follow-up: Power of Three (#326) — no bit trick; use 3^19=1162261467 divisibility.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: Power of Four (#342) — power of 2 AND set bit in odd position.

IN YOUR APP | Expose | Editorial | Community

EASY | Bit Manipulation | LeetCode #268 | 7 of 12 | 7/12

Missing Number

QUESTION

Given an array nums containing n distinct numbers in the range [0, n], return the only number in the range that is missing from the array.

Example 1	
Input	`nums = [3,0,1]`
Output	`2`
Explanation	*n = 3 since there are 3 numbers, so all numbers are in the range [0,3]. 2 is the missing number in the range since it does not appear in nums.*

Example 2	
Input	`nums = [0,1]`
Output	`2`
Explanation	*n = 2 since there are 2 numbers, so all numbers are in the range [0,2]. 2 is the missing number in the range since it does not appear in nums.*

Example 3	
Input	`nums = [9,6,4,2,3,5,7,0,1]`
Output	`8`
Explanation	*n = 9 since there are 9 numbers, so all numbers are in the range [0,9]. 8 is the missing number in the range since it does not appear in nums.*

CONSTRAINTS

- n == nums.length
- $1 <= n <= 10^4$
- 0 <= nums[i] <= n
- All the numbers of nums are unique.
- Follow up: Could you implement a solution using only O(1) extra space complexity and O(n) runtime complexity?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Array of n distinct numbers from [0,n]. Exactly one number is missing. Find it. Right?"

"A few quick questions:

Array length is n (not n+1)? Yes — one number from [0,n] is absent.

Can 0 be missing? Yes. Can n itself be missing? Yes."

PHASE 1 CLARIFY

"Let me walk: nums=[3,0,1] -> 2 missing ok.
nums=[9,6,4,2,3,5,7,0,1] -> 8 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for v in range(0,_n_+_1):
    scan the array for v
return the first v not found
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

A scan per candidate. The expected sum n(n+1)/2 minus the actual sum (or XOR with all indices) is O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sum formula: n(n+1)/2 minus actual sum — O(n). Walk-through:"*

`expected = n*(n+1)//2 (sum of 0..n).`

`actual = sum(nums).`

Return expected - actual.

WHY Gauss formula: sum 0..n = n*(n+1)/2. Array is missing exactly one of these.

The difference = the missing number.

Alternative: XOR all 0..n with all elements -> XOR pairs cancel -> missing number remains.

XOR: result = 0. For i in 0..n: result ^= i. For num in nums: result ^= num. Return result.

Both O(n) time, O(1) space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _268_MissingNumber:
    def missingNumber(self, nums):
        result = len(nums)
        for i, num in enumerate(nums):
            result ^= i ^ num
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the bit representation and show which bits get set/cleared and why."

nums=[3,0,1], n=3:

result=3. i=0: 3^0^3=0. i=1: 0^1^0=1. i=2: 1^2^1=2. return 2 ok

PHASE 5 TEST & DEBUG

nums=[0,1]: n=2. result=2. i=0: 2^0^0=2. i=1: 2^1^1=2. return 2 ok

XOR cancels every pair (i, nums[i]) except for the missing index

PHASE 6 COMPLEXITY & FOLLOW-UP

Sum formula O(n): simple but may overflow in Java/C++ for large n.

XOR O(n): overflow-safe, same asymptotic.

Follow-up: Find Duplicate Number (#287) — one value appears twice; Floyd's cycle detection.

Follow-up: First Missing Positive (#41) — different range, needs cyclic-sort approach."

Space: O(n) — proportional to input or output size; O(1) extra if in-place.

EASY | Bit Manipulation | LeetCode #338 | 8 of 12 | 8/12

Counting Bits

QUESTION

Given an integer n, return an array ans of length n + 1 such that for each i (0 <= i <= n), ans[i] is the number of 1's in the binary representation of i.

Example 1	
Input	`n = 2`
Output	`[0,1,1]`
Explanation	*0 --> 0* *1 --> 1* *2 --> 10*

Example 2	
Input	`n = 5`
Output	`[0,1,1,2,1,2]`
Explanation	*0 --> 0* *1 --> 1* *2 --> 10* *3 --> 11* *4 --> 100* *5 --> 101*

CONSTRAINTS

- $0 <= n <= 10^5$
- Follow up:
- It is very easy to come up with a solution with a runtime of O(n log n). Can you do it in linear time O(n) and possibly in a single pass?
- Can you do it without using any built-in function (i.e., like __builtin_popcount in C++)?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return ans[i] = number of 1-bits in i, for all i in [0, n]. Length n+1. Right?"

"A few quick questions:

n can be 0? Yes — ans=[0]. Must run in O(n) time? Yes per follow-up."

"Let me walk: n=2 -> [0,1,1] ok. n=5 -> [0,1,1,2,1,2] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(0,_n_+_1):
    count its 1-bits by checking each bit
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n log n)	O(1)

WHY IT IS SLOW

Recounts every number from scratch. dp[i] = dp[i >> 1] + (i & 1) reuses an earlier answer: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is counting bits independently for each number.

DP with bit trick: ans[i] = ans[i >> 1] + (i & 1).

Shifting right by 1 drops the LSB (Least Significant Bit) — we already know that count. Add 1 if LSB was set.

Time: O(n). Space: O(n) for the output array.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _338_CountingBits:
    def countBits(self, n):
        ans = [0] * (n + 1)
        for i in range(1, n + 1):
            ans[i] = ans[i >> 1] + (i & 1)
        return ans
```

PHASE 5 TEST & DEBUG

"Let me trace countBits(5)."

dp[0]=0, dp[1]=1, dp[2]=dp[2>>1]+2&1=dp[1]+0=1, dp[3]=dp[1]+1=2, dp[4]=dp[2]+0=1, dp[5]=dp[2]+1=2

Result: [0,1,1,2,1,2] ok

WHY dp[i]=dp[i>>1]+(i&1): i and i>>1 differ only by the last bit; shifting right removes one bit.

No bit-counting loop needed; O(n) time, O(n) space for output array.

Verify: 5=101(binary)->2 ones; dp[5]=2 ok. 4=100->1 one; dp[4]=1 ok.

Edge: n=0 -> return [0].

Verify: 7=111(binary)->3 ones; dp[7]=dp[3]+1=2+1=3 ok. Recurrence leverages shift property.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) one pass, O(1) per entry	O(n) output only

Alternative: ans[i] = ans[i & (i-1)] + 1 (clear lowest set bit then add 1).

Follow-up: Number of 1 Bits (#191) — single number, Brian Kernighan O(k) per call.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: Total Hamming Distance (#477) — use column-wise bit counting across all pairs.

IN YOUR APP | Expose | Editorial | Community

EASY | Bit Manipulation | LeetCode #461 | 9 of 12 | 9/12

Hamming Distance

QUESTION

The Hamming distance between two integers is the number of positions at which the corresponding bits are different.

Given two integers x and y, return the Hamming distance between them.

Example 1

Input	`x = 1, y = 4`
Output	`2`
Explanation	*1 (0 0 0 1)* *4 (0 1 0 0)* *The above arrows point to positions where the corresponding bits are different.*

Example 2

Input	`x = 3, y = 1`
Output	`1`

CONSTRAINTS

- $0 <= x, y <= 2^{31} - 1$
- Note: This question is the same as 2220: Minimum Bit Flips to Convert Number.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the number of bit positions where x and y differ. Right?"

"A few quick questions:

XOR gives 1 at positions where bits differ; count set bits in XOR? Yes."

"Let me walk: x=1(001), y=4(100) -> XOR=101 -> 2 differing bits ok

PHASE 2 BRUTE FORCE (also optimal)

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
turn x and y into binary strings
count the positions where the characters differ
```

TIME	SPACE
O(log n)	O(log n)

NOTE

PHASE 2 BRUTE FORCE (also optimal)

Not slow: about 31 bits. Strings are unnecessary: XOR the numbers and count the set bits (n & (n - 1) removes one each time) in O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"XOR then count set bits — O(1). Walk-through:"

`diff = x ^ y <- bits that DIFFER between x and y are set to 1; bits that are the same = 0.`

Hamming distance = popcount(diff) = number of 1-bits in diff.

`bin(diff).count('1') in Python gives the bit count directly.`

Alternative: Brian Kernighan's algorithm: while diff: diff &= (diff-1); count++.

Each iteration clears the lowest set bit — O(Hamming distance) iterations.

WHY XOR: bits that differ between x and y -> 1. Those that are same -> 0. Count the 1s.

Time: O(1) for 32-bit integers. Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _461_HammingDistance:
    def hammingDistance(self, x, y):
        diff = x ^ y
        count = 0
        while diff:
            diff &= diff - 1
            count += 1
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace hammingDistance(1, 4)."

1 = 001, 4 = 100

XOR: 001 ^ 100 = 101

Count 1-bits: bin(5) = '101' -> 2 ones -> return 2 ok

WHY XOR: positions where bits differ become 1; count of 1s = hamming distance.

Trace (1,3): 01 ^ 11 = 10 -> 1 one -> return 1 ok

Edge: same number -> XOR=0, distance=0. All bits differ -> XOR=all 1s.

`popcount = bin(x ^ y).count('1') in Python.`

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(1) at most 32 set bits	O(1)

PHASE 6 COMPLEXITY & FOLLOW-UP

`bin(x^y).count('1') is a one-liner alternative — same O(1).`

Follow-up: Total Hamming Distance (#477) — for each bit position, count 0s*1s across all pairs.

Follow-up: Minimum Bit Flips to Convert Number (#2220) — same as Hamming distance.

IN YOUR APP | Expose | Editorial | Community

EASY | Bit Manipulation | LeetCode #645 | 10 of 12 | 10/12

Set Mismatch

QUESTION

You have a set of integers s, which originally contains all the numbers from 1 to n. Unfortunately, due to some error, one of the numbers in s got duplicated to another number in the set, which results in repetition of one number and loss of another number.

You are given an integer array nums representing the data status of this set after the error.

Find the number that occurs twice and the number that is missing and return them in the form of an array.

Example 1	
Input	`nums = [1,2,2,4]`
Output	`[2,3]`

Example 2	
Input	`nums = [1,1]`
Output	`[1,2]`

CONSTRAINTS

- 2 <= nums.length <= 10^4
- 1 <= nums[i] <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Array should be [1..n] but one number is missing and one appears twice.

Find [duplicate, missing]. Right?"

"A few quick questions:

Length of nums is n? Yes. Can solve with math or bit manipulation in O(n) O(1). Right."

"Let me walk: nums=[1,2,2,4] -> duplicate=2, missing=3 -> [2,3] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for v in range(1,_n_+_1):
    count how often it occurs by scanning the array
    the value with count 2 is the duplicate,
        count 0 the missing one
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

A scan per value. Mark each value by negating nums[abs(v) - 1] (a second visit finds the duplicate; the positive index left is the missing one): O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"XOR or math to find missing and duplicate — O(n). Walk-through:"

Expected XOR = 1^2^...^n. Actual XOR = XOR of all nums.

XOR of both = missing ^ duplicate (all others cancel).

But to find them separately: use the set-bit trick (like #260).

OR simpler: math approach:

`sum_expected = n*(n+1)//2. sum_actual = sum(nums).`

`sum_diff = sum_actual - sum_expected = duplicate - missing.`

`sq_expected = n*(n+1)*(2n+1)//6. sq_actual = sum(x^2 for x in nums).`

`sq_diff = sq_actual - sq_expected = duplicate² - missing².`

Solve: duplicate = (sq_diff/sum_diff + sum_diff) / 2. missing = duplicate - sum_diff.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _645_SetMismatch:
    def findErrorNums(self, nums):
        from collections import Counter
        count = Counter(nums)
        n = len(nums)
        duplicate = next(v for v, c in count.items() if c
== 2)
        missing = next(i for i in range(1, n + 1) if count
[i] == 0)
        return [duplicate, missing]
```

PHASE 5 TEST & DEBUG

"Let me trace findErrorNums([1,2,2,4])."

XOR approach: XOR all nums with 1..n

1^2^2^4 ^ 1^2^3^4 = 2^3 = 1? Let's use sum approach instead

`sum([1,2,2,4])=9, expected=1+2+3+4=10 -> diff=1 -> dup-missing=1`

`sum(set([1,2,2,4]))=7, expected=10 -> missing=10-7=3; dup=missing-diff=3-1=2? No`

dup=sum-sum_no_dup? sum([1,2,2,4])=9, sum(set)=1+2+4=7 -> dup appears extra: 9-7=2 (duplicate=2)

missing=expected-sum_no_dup=10-7=3

Return [2,3] ok

WHY sum + set: O(n) with simple arithmetic. dup = sum-set_sum, missing = expected-set_sum.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** Counter

O(1) space: XOR approach or mark-negative approach (like #448).

Follow-up: Find All Duplicates in an Array (#442) — multiple duplicates; mark-negative.

Follow-up: Missing Number (#268) — only missing, no duplicate; XOR or sum.

IN YOUR APP | Expose | Editorial | Community

EASY | Bit Manipulation | LeetCode #1763 | 11 of 12 | 11/12

Longest Nice Substring

QUESTION

A string s is nice if, for every letter of the alphabet that s contains, it appears both in uppercase and lowercase. For example, "abABB" is nice because 'A' and 'a' appear, and 'B' and 'b' appear. However, "abA" is not because 'b' appears, but 'B' does not.

Given a string s, return the longest substring of s that is nice. If there are multiple, return the substring of the earliest occurrence. If there are none, return an empty string.

Example 1

Input	`s = "YazaAay"`
Output	`"aAa"`
Explanation	*"aAa" is a nice string because 'A/a' is the only letter of the alphabet in s, and both 'A' and 'a' appear. "aAa" is the longest nice substring.*

Example 2

Input	`s = "Bb"`
Output	`"Bb"`
Explanation	*"Bb" is a nice string because both 'B' and 'b' appear. The whole string is a substring.*

Example 3

Input	`s = "c"`
Output	`""`
Explanation	*There are no nice substrings.*

CONSTRAINTS

- 1 <= s.length <= 100
- s consists of uppercase and lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

A nice substring has both uppercase and lowercase of each letter.

Find the longest nice substring. If no nice substring, return ''. Right?"

"A few quick questions:

If multiple of same max length, return any? Yes. String contains only letters? Yes."

"Let me walk: s='YazaAay' -> 'aAa' or 'Aa' or 'aA'... wait 'aAa' has a and A ok -> 'aAa' length 3? Actually 'YazaAay' -> 'aAa' appears? No. 'aAa' is not a substring... 'azaAay' contains a/A but z has no Z... -> '' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for substring in substrings:
    check that every letter present appears in
        both cases
    best = max(best, (earliest_on_ties))
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

O(n^2) substrings, each checked in O(n). Divide and conquer: split at any letter whose other case is missing (it cannot be inside a nice substring) and recurse on the pieces.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Divide and conquer: check if substring is nice, split on non-nice chars — O(n²). Walk-through:"

A 'nice' string has both uppercase and lowercase of each character present.

`def longestNiceSubstring(s):`

For c in s:

If c.swapcase() not in s: <- this char breaks the nice property

`parts = s.split(c) (split on this 'bad' char).`

Return longest of [longestNiceSubstring(part) for part in parts].

`return s (all chars are nice -> whole string is nice)`

WHY split on bad char: any nice substring cannot contain a char without its pair.

So the longest nice substring lies entirely within one of the splits.

WHY recurse on parts: each split may itself have bad characters -> recurse to find nice.

Time: O(n²) worst case. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1763_LongestNiceSubstring:
    def longestNiceSubstring(self, s):
        if len(s) < 2:
            return ''
        chars = set(s)
        for i, c in enumerate(s):
            if c.lower() not in chars or c.upper() not in
chars:
                left = self.longestNiceSubstring(s[:i])
                right = self.longestNiceSubstring(s[i+1:])
                return left if len(left) >= len(right) els
e right
        return s
```

PHASE 5 TEST & DEBUG

"Let me trace the bit representation and show which bits get set/cleared and why."

s='YazaAay': Y has no y -> split at Y. left='' (empty). right=longestNice('azaAay').

'azaAay': z has no Z -> split. left=longestNice('a')=''. right=longestNice('aAay').

'aAay': all letters have both cases? a/A present, y has no Y -> split at y. left='aAa' ok

```
return 'aAa' (longer than '') -> answer='aAa' ok
```

Returning left when equal length gives consistent results for ties

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^2)$	**O(n)**
at most n splits, each O(n) scan	recursion

Follow-up: count all nice substrings — accumulate count instead of max length.

Follow-up: if Unicode characters allowed, use set membership instead of upper/lower.

IN YOUR APP | Expose | Community

EASY | Bit Manipulation | LeetCode #1863 | 12 of 12 | 12/12

Sum of All Subset XOR Totals

QUESTION

The XOR total of an array is defined as the bitwise XOR of all its elements, or 0 if the array is empty.

CONSTRAINTS

- For example, the XOR total of the array [2,5,6] is 2 XOR 5 XOR 6 = 1.

Given an array nums, return the sum of all XOR totals for every subset of nums.

Note: Subsets with the same elements should be counted multiple times.

An array a is a subset of an array b if a can be obtained from b by deleting some (possibly zero) elements of b.

Example 1

Input	`nums = [1,3]`
Output	`6`
Explanation	*The 4 subsets of [1,3] are:* *- The empty subset has an XOR total of 0.* *- [1] has an XOR total of 1.* *- [3] has an XOR total of 3.* *- [1,3] has an XOR total of 1 XOR 3 = 2.* *0 + 1 + 3 + 2 = 6*

Example 2

Input	`nums = [5,1,6]`
Output	`28`
Explanation	*The 8 subsets of [5,1,6] are:* *- The empty subset has an XOR total of 0.* *- [5] has an XOR total of 5.* *- [1] has an XOR total of 1.* *- [6] has an XOR total of 6.* *- [5,1] has an XOR total of 5 XOR 1 = 4.* *- [5,6] has an XOR total of 5 XOR 6 = 3.* *- [1,6] has an XOR total of 1 XOR 6 = 7.* *- [5,1,6] has an XOR total of 5 XOR 1 XOR 6 = 2.* *0 + 5 + 1 + 6 + 4 + 3 + 7 + 2 = 28*

Example 3

Input	`nums = [3,4,5,6,7,8]`
Output	`480`
Explanation	*The sum of all XOR totals for every subset is 480.*

- 1 <= nums.length <= 12
- 1 <= nums[i] <= 20

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

For every subset of nums, compute the XOR of its elements (XOR total, 0 for empty subset). Sum all these XOR totals. Right?"

"A few quick questions:

nums length is small (<=12 per constraints), so brute force over all 2^n subsets is feasible? Yes."

"Let me walk: nums=[1,3] -> subsets: {}=0,{1}=1,{3}=3,{1,3}=2 -> sum=6 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subset in subsets(nums):
    XOR its elements
    add up the totals
```

TIME	SPACE
O(n * 2^n)	O(1)

WHY IT IS SLOW

Enumerates 2^n subsets. A bit contributes to exactly half of all subsets if any element has it, so the answer is (OR of all numbers) * 2^(n - 1): O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(2^n) enumeration. There's a math shortcut: each bit position set in ANY number contributes to exactly

HALF of all 2^n subsets (since for each such bit, half the subsets include an odd count of numbers with

that bit set). So the answer is simply (OR of all nums) << (n-1) — the OR captures which bits ever appear,

and each contributes in 2^(n-1) subsets.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1863_SumOfAllSubsetXORTotals:
    def subsetXORSum(self, nums):
        or_all = 0
        for num in nums:
            or_all |= num
        return or_all << (len(nums) - 1)
```

PHASE 5 TEST & DEBUG

"Let me trace subsetXORSum([1,3])."

All subsets: [], [1], [3], [1,3]

XOR totals: 0, 1, 3, 1^3=2

Sum: 0+1+3+2=6 ok

Pattern: each element contributes to 2^(n-1) subsets (half of all 2^n subsets)

Result = (OR of all elements) * 2^(n-1)

[1,3]: OR=1|3=3. 3 * 2^(2-1) = 3*2=6 ok

WHY bit-level: each bit of OR contributes exactly 2^(n-1) to the sum (half subsets have it set).

Edge: single element -> result = element (one subset [elem], XOR=elem; empty subset XOR=0).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**

Follow-up: if asked to prove the formula, explain the bit-parity argument: for a fixed bit, half of all subsets

have an odd count of elements with that bit, contributing the bit's value.

Follow-up: Single Number (#136) — related bit-counting intuition for XOR over a single array.

IN YOUR APP | Expose | Editorial | Community